# Bölüm 6: String'ler

SE 422 · Introduction to Data Science · Konu özeti

Kaynak slayt destesi: week4_MoreStrings
7 Kasım 2026

# İçindekiler

**Bölüm 6 · String'ler . . . . . 3**

Bölümün amacı . . . . . 3

Konu anlatımı . . . . . 3

String bir dizidir (Slayt 1–7) . . . . . 3

Birleştirme ve değişmezlik (Slayt 8–10) . . . . . 4

Dilimleme ve in (Slayt 11–12) . . . . . 4

Test metotları (Slayt 13–14) . . . . . 4

Değiştirme metotları (Slayt 15–16) . . . . . 5

Arama ve değiştirme (Slayt 17–19) . . . . . 5

Tekrar operatörü (Slayt 20) . . . . . 6

split ve token'lar (Slayt 21–27) . . . . . 6

Sık yapılan hatalar . . . . . 6

Kısa tekrar soruları . . . . . 7

Cevaplar . . . . . 7

# Bölüm 6 · String'ler

## Bölümün amacı

Programların büyük kısmı metinle çalışır. Bu bölüm string'leri incelemeyi ve dönüştürmeyi öğretir. Sonunda şunları yapabilmelisin:

- String'in karakterlerine döngü ve indeksle erişmek; uzunluğunu bulmak
- Birleştirme ve tekrar operatörlerini kullanmak; string'lerin neden değiştirilemez olduğunu açıklamak
- Dilimleme, `in` ve test/değiştirme/arama metotlarıyla çalışmak
- `split` ile string'i token'lara ayırmak

## Konu anlatımı

### String bir dizidir (Slayt 1–7)

String'ler **dizidir**; bu yüzden listelerde öğrendiğin araçların çoğu string'lerde de çalışır. Karakterlere iki yolla erişilir:

- **for döngüsü:** tüm karakterler üzerinde dolaşmak için (ör. bir harfi saymak)
- **İndeks:** `s[i]`, 0'dan başlar; negatif indeks sondan sayar. Aralık dışı indeks `IndexError` verir, `len(s)` uzunluğu döndürür.

PYTHON

```
name = 'Juliet'
for ch in name:
    print(ch, end=' ')
print()
my_string = 'Roses are red'
print(my_string[6], my_string[-1], len(my_string))
sayac = 0
for ch in 'banana':
    if ch == 'a':
        sayac += 1
print(sayac)
```

ÇIKTI

```
J u l i e t
a d 13
3
```

## Birleştirme ve değişmezlik (Slayt 8–10)

`+` ve `+=` string'leri birleştirir. String'ler **değiştirilemez** (immutable): birleştirme var olan string'i değiştirmez, **yeni bir string** oluşturup değişkene atar. `s[0] = 'X'` gibi bir atama istisna fırlatır.

**PYTHON**

```
name = 'Carmen'
name = name + ' Brown'
letters = 'abc'
letters += 'def'
print(name, '|', letters)
```

**ÇIKTI**

```
Carmen Brown | abcdef
```

**PYTHON**

```
s = 'python'
s[0] = 'P'
```

**ÇIKTI**

```
TypeError: 'str' object does not support item assignment
```

## Dilimleme ve in (Slayt 11–12)

`s[start:end]` bir **alt string** döndürür (end dahil değil). Başlangıç yoksa 0, bitiş yoksa uzunluk; adım ve negatif indeks kullanılabilir. `in` / `not in` bir string'in diğerinde geçip geçmediğini sınar.

**PYTHON**

```
full_name = 'Patty Lynn Smith'
print(full_name[0:5], '|', full_name[6:], '|', full_name[-5:], '|',
↪ full_name[::-1])
print('Lynn' in full_name, 'Ann' not in full_name)
```

**ÇIKTI**

```
Patty | Lynn Smith | Smith | htimS nnyL yttaP
True True
```

## Test metotları (Slayt 13–14)

`s.metot()` biçimindeki bu metotlar True/False döndürür:

| Metot | True, eğer string… |
|---|---|
| `isalnum()` | yalnızca harf veya rakam içeriyorsa |
| `isalpha()` | yalnızca harf içeriyorsa |

| Metot | True, eğer string… |
|---|---|
| `isdigit()` | yalnızca rakam içeriyorsa |
| `islower()` / `isupper()` | tüm harfleri küçük / büyükse |
| `isspace()` | yalnızca boşluk karakterleri içeriyorsa |

PYTHON
```
print('abc123'.isalnum(), 'abc'.isalpha(), '2024'.isdigit())
print('Ali'.isupper(), 'ali'.islower(), '  \n'.isspace())
```

ÇIKTI
```
True True True
False True True
```

## Değiştirme metotları (Slayt 15–16)

Bu metotlar string'i değiştirmez, **değiştirilmiş bir kopya** döndürür: `lower()`, `upper()`, `strip()` / `lstrip()` / `rstrip()` (uçlardaki boşlukları ya da verilen karakteri siler). Karşılaştırma büyük/küçük harfe duyarlı olduğundan kullanıcı girdisi genellikle `strip().lower()` ile normalize edilir.

PYTHON
```
cevap = '  Evet  '
print(cevap.strip().lower() == 'evet')
print('$$100$$'.strip('$'), 'python'.upper(), repr('  ab  '.rstrip()))
```

ÇIKTI
```
True
100 PYTHON '  ab'
```

## Arama ve değiştirme (Slayt 17–19)

`startswith` / `endswith` başlangıç ve bitişi sınar. `find(alt)` alt string'in ilk indeksini, bulamazsa **-1** döndürür. `replace(eski, yeni)` her geçişi değiştirilmiş bir kopya üretir.

PYTHON
```
filename = 'rapor.txt'
print(filename.endswith('.txt'), filename.startswith('rap'))
string = 'Four score and seven years ago'
print(string.find('seven'), string.find('ten'))
print(string.replace('years', 'days'))
```

ÇIKTI

```
True True
15 -1
Four score and seven days ago
```

### Tekrar operatörü (Slayt 20)

`string * n` string'in n kopyasını birleştirir; string solda, sayı sağdadır.

PYTHON

```python
print('Ha' * 3)
print('-' * 20)
```

ÇIKTI

```
HaHaHa
--------------------
```

### split ve token'lar (Slayt 21–27)

`split()` string'i bir **listeye** böler; varsayılan ayırıcı boşluktur, başka bir **sınırlayıcı** (delimiter) argüman olarak verilebilir. Sınırlayıcıyla ayrılmış parçalara **token**, bunları ayırma işine **tokenizing** denir.

PYTHON

```python
print('One two three four'.split())
print('1/2/3/4/5'.split('/'))
tokens = 'www.example.com'.split('.')
print(tokens)
sayilar = '17;92;81;12;46;5'.split(';')
print(sayilar, sum(int(t) for t in sayilar))
```

ÇIKTI

```
['One', 'two', 'three', 'four']
['1', '2', '3', '4', '5']
['www', 'example', 'com']
['17', '92', '81', '12', '46', '5'] 253
```

## Sık yapılan hatalar

- **String'i yerinde değiştirmeye çalışmak:** `s[0] = 'P'` → `TypeError`; yeni string oluştur: `'P' + s[1:]`.
- **Metotların string'i değiştirdiğini sanmak:** `s.upper()` tek başına `s`'yi değiştirmez; `s = s.upper()` yazılmalı.

PYTHON

```
s = 'merhaba'
s.upper()
print(s)
s = s.upper()
print(s)
```

ÇIKTI

```
merhaba
MERHABA
```

- `find` **sonucunu yanlış yorumlamak:** bulunamayınca hata değil `-1` döner; `if s.find(x):` yanlıştır (indeks 0 da False sayılır), `if x in s:` kullan.
- **Var olmayan metot çağırmak:** string'lerin `reverse()` metodu yoktur → `AttributeError`; tersi için `s[::-1]`.
- **Token'ı sayıya çevirmeden toplamak:** `split` her zaman string listesi döndürür.

## Kısa tekrar soruları

1 `'Python'[1:4]` ve `'Python'[-3:]` nedir?

2 `name = 'Ada'; name += ' Lovelace'` satırı var olan string'i mi değiştirir?

3 `'hello world'.find('o')` ve `'hello'.find('z')` ne döndürür?

4 `' Merhaba '.strip().upper()` nedir?

5 `'a,b,,c'.split(',')` nedir?

6 `'3' * 3` ile `3 * 3` farkı nedir?

## Cevaplar

1 `'yth'` ve `'hon'`.

2 Hayır; string'ler değiştirilemez. Yeni bir `'Ada Lovelace'` string'i oluşur ve `name` onu gösterir.

3 `4` (ilk geçtiği indeks) ve `-1`.

4 `'MERHABA'`

5 `['a', 'b', '', 'c']` (iki ayraç arasındaki boş token korunur).

6 `'3' * 3` → `'333'` (tekrar), `3 * 3` → `9` (çarpma).